

Compte Rendu - Application Web de Gestion de Notes Personnelles

Module	CPI2 – Programmation Web
Enseignant	Hamdi Nasreddine
Etudiants	Adem Fattouch – Iyed Abaza
Technologies	Laravel 11 + React 18 + Tailwind CSS
Authentification	Laravel Sanctum (tokens Bearer)
Base de données	MySQL

1. Introduction

Ce projet a pour objectif de développer une application web permettant à un utilisateur de gérer ses notes personnelles. L'application offre des fonctionnalités d'authentification (inscription et connexion) ainsi que la gestion complète des notes (création, consultation, modification et suppression).

2. Architecture technique

L'application repose sur une architecture client-serveur séparée en trois parties principales : le frontend, le backend et la base de données.

Le frontend est développé avec React. Il représente la partie visible pour l'utilisateur et permet d'interagir avec l'application (connexion, ajout de notes, modification, suppression, etc.). Les actions de l'utilisateur déclenchent des requêtes HTTP envoyées via Axios.

Ces requêtes sont envoyées vers une API REST développée avec Laravel. Le backend traite les demandes, applique la logique métier (validation des données, gestion des utilisateurs, gestion des notes) et assure la sécurité grâce à l'authentification par token avec Laravel Sanctum.

Le backend communique ensuite avec une base de données (MySQL ou SQLite) pour stocker et récupérer les informations des utilisateurs et des notes.

Le flux de communication peut être résumé comme suit :

React (client) → Axios → API Laravel → Base de données
Base de données → API Laravel → React (réponse JSON)

Cette architecture permet une séparation claire entre les différentes couches de l'application.

3. Captures commentées

The image shows two side-by-side screenshots of the NoteFlow application's authentication pages. The left screenshot is the 'Page De Login' (Login Page). It features the NoteFlow logo at the top, a 'Welcome back' message, and a form with fields for 'Email' (containing 'you@example.com') and 'Password' (masked with dots). A purple 'Sign in' button is at the bottom of the form. Below the form, there is a link: 'Don't have an account? [Create one](#)'. The right screenshot is the 'Page D'inscription' (Registration Page). It also features the NoteFlow logo and the text 'Create your account'. The form includes fields for 'Name' (containing 'Your name'), 'Email' (containing 'you@example.com'), and 'Password' (masked with dots). A purple 'Create Account' button is at the bottom of the form. Below the form, there is a link: 'Already have an account? [Sign in](#)'.

Page De Login

Page D'inscription

The image shows a screenshot of the NoteFlow application's main dashboard. At the top, there is a header with the NoteFlow logo on the left and a 'Sign out' button on the right. Below the header, there is a section with four colored buttons: 'Total 3' (black), 'High 1' (red), 'Medium 1' (yellow), and 'Low 1' (green). Below this, there is a search bar with the placeholder text 'Search notes...'. To the right of the search bar, there are four buttons: 'All 3' (black), 'High' (red), 'Medium' (yellow), and 'Low' (green). Below the search bar and filters, there are three note cards. The first card is titled 'La vie rose' and has a 'Low' priority tag. The second card is titled 'Note 2' and has a 'Medium' priority tag. The third card is titled 'First Note' and has a 'High' priority tag. Each card shows the note's content, a date '10 mai 2026', and 'Edit' and 'Delete' buttons. On the left side of the dashboard, there is a 'New Note' form with fields for 'Title *' (containing 'La vie rose') and 'Content' (containing '-----'). Below the form, there are three priority buttons: 'Low' (green), 'Medium' (yellow), and 'High' (red). A purple 'Add Note' button is at the bottom of the form.

Accueil Du page présente une zone de recherche, filtre de priorité et les notes, l'ajout d'une note...

New Note

Title *

New note !

Content

Vide.

Priority

Low

Medium

High

Add Note

Q Search notes...

All 3HighMediumLow

La vie roseLow

10 mai 2026EditDelete

Note 2Medium

10 mai 2026EditDelete

First NoteHigh

10 mai 2026EditDelete

Avant l'ajout

NoteFlowSign out

Total 4High 1Medium 1Low 2

New Note

Title *

New note !

Content

Vide.

Priority

Low

Medium

High

Add Note

Q Search notes...

All 4HighMediumLow

New note !Low

10 mai 2026EditDelete

La vie roseLow

10 mai 2026EditDelete

Note 2Medium

10 mai 2026EditDelete

First NoteHigh

10 mai 2026EditDelete

Note Ajoutée

Total 3High 1Medium 1Low 1

New Note

Title *

New note !

Content

Vide.

Priority

Low

Medium

High

Add Note

Q Search notes...

All 3HighMediumLow

New note !Low

10 mai 2026EditDelete

Note 2Medium

10 mai 2026EditDelete

First NoteHigh

10 mai 2026EditDelete

Après La suppression d'une note

4. Extraits de code

NoteController.php

```
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Models>Note;
6  use Illuminate\Http\Request;
7
8  class NoteController extends Controller
9  {
10     public function index(Request $request)
11     {
12         $user = $request->user();
13
14         return Note::where('user_id', $user->id)
15             ->orderBy('created_at', 'desc')
16             ->get();
17     }
18
19     public function store(Request $request)
20     {
21         $request->validate([
22             'title' => 'required|max:100',
23             'content' => 'nullable',
24             'priority' => 'required|in:low,medium,high',
25         ]);
26
27         $note = Note::create([
28             'title' => $request->title,
29             'content' => $request->content,
30             'priority' => $request->priority,
31             'user_id' => $request->user()->id,
32         ]);
33
34         return response()->json([
35             'message' => 'Note created',
36             'note' => $note
37         ]);
38     }
39
40     // UPDATE note
41     public function update(Request $request, $id)
42     {
43         $note = Note::findOrFail($id);
44
45         if ($note->user_id !== $request->user()->id) {
46             return response()->json(['message' => 'Unauthorized'], 403);
47         }
48
49         $note->update($request->only(['title', 'content', 'priority']));
50     }
```

```

51         return response()->json([
52             'message' => 'Note updated',
53             'note' => $note
54         ]);
55     }
56
57     public function destroy(Request $request, $id)
58     {
59         $note = Note::findOrFail($id);
60
61         if ($note->user_id !== $request->user()->id) {
62             return response()->json(['message' => 'Unauthorized'], 403);
63         }
64
65         $note->delete();
66
67         return response()->json([
68             'message' => 'Note deleted'
69         ]);
70     }
71 }

```

Notes.jsx

```

40     const handleSubmit = async (data) => {
41         setSubmitting(true);
42         try {
43             if (editingNote) {
44                 await notesService.update(token, editingNote.id, data);
45                 setEditingNote(null);
46             } else {
47                 await notesService.create(token, data);
48             }
49             await fetchNotes();
50         } finally {
51             setSubmitting(false);
52         }
53     };

```

NoteServices.js

```

import api from "../api";

const authHeader = (token) => ({ headers: { Authorization: `Bearer ${token}` } });

export const notesService = {
    getAll: (token) => api.get("/notes", authHeader(token)),
    create: (token, data) => api.post("/notes", data, authHeader(token)),
    update: (token, id, data) => api.put(`/notes/${id}`, data, authHeader(token)),
    delete: (token, id) => api.delete(`/notes/${id}`, authHeader(token)),
};

```

Api.js

```
1 import axios from "axios";
2
3 const api = axios.create({
4   |   baseURL: "http://127.0.0.1:8000/api",
5   | });
6
7 export default api;
8 |
```

api.php

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\AuthController;
use App\Http\Controllers>NoteController;

Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);

Route::middleware('auth:sanctum')->group(function () {

    Route::post('/logout', [AuthController::class, 'logout']);

    Route::get('/notes', [NoteController::class, 'index']);
    Route::post('/notes', [NoteController::class, 'store']);
    Route::put('/notes/{id}', [NoteController::class, 'update']);
    Route::delete('/notes/{id}', [NoteController::class, 'destroy']);

});
```

5. Difficultés & solutions

Plusieurs difficultés ont été rencontrées :

- Gestion du token d'authentification
- Problèmes CORS entre frontend et backend
- Mise à jour de la liste après suppression

Des solutions ont été mises en place en configurant correctement les headers et en rafraîchissant les données après chaque action.

6. Tests manuels

L'ajout

New Note

Title *

AAAAA

Content

Vide.

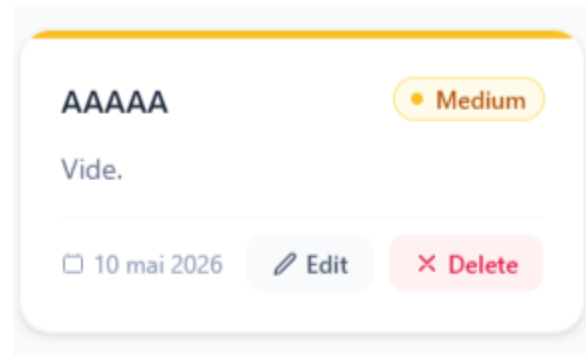
Priority

Low

Medium

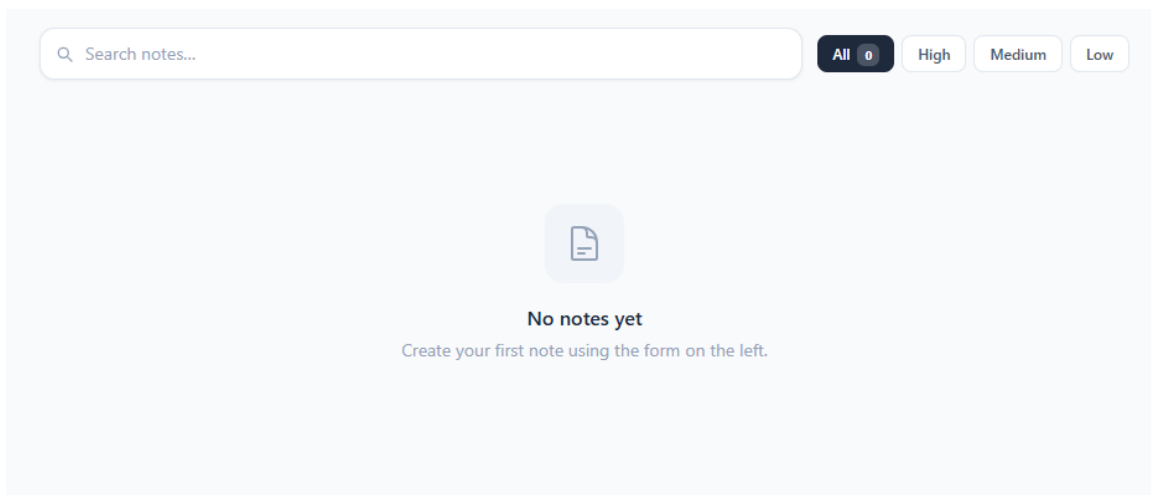
High

Add Note



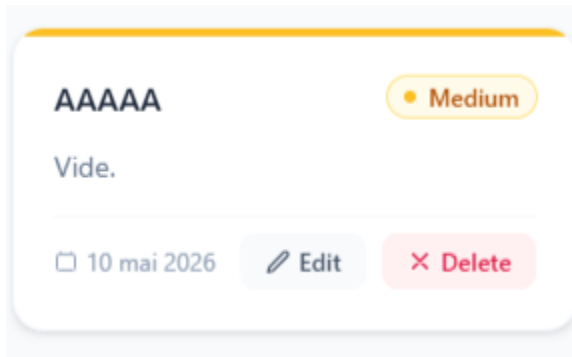
Note Ajoutée

Suppression (On va supprimée la note qui on a déjà ajoutée) :

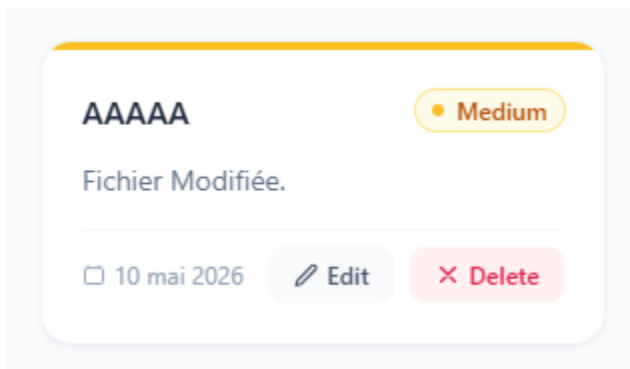


Note Supprimée

Modification :



Note avant modification



Note Après Modification

7. Bilan & évolutions

Ce projet a permis de renforcer les compétences en développement web full-stack. Des améliorations possibles incluent l'ajout de filtres, d'un système de recherche, ou encore le partage de notes entre utilisateurs.

8. Lien GitHub vers le Projet

🔗 Dépôt GitHub du projet :

<https://github.com/adamfattouch/NoteApp>